FUNDAMENTAL OF AI & ML – VITYARTHI PROJECT

~ Shaurya Pratap Singh

CONCEPT USED:

1. State Space Search:

At its core, the city is modeled as a **Graph**.

- **Nodes (S):** Represent intersections or locations (A, B, C, D).
- **Edges (E):** Represent the roads connecting them.
- **Weights (W):** Represent the cost (distance/time) to travel between nodes.

The AI's task is to navigate this "State Space" to find a sequence of actions that leads from the *Initial State* to the *Goal State*.

2. Uninformed (Blind) Search:

These algorithms explore the map without any "hunch" about where the goal is.

- **Breadth-First Search (BFS):** Explores all neighbor nodes at the present depth before moving to nodes at the next depth level.
    - *Concept:* It uses a **FIFO (First-In-First-Out) Queue**.
    - *Outcome:* It guarantees the shortest path in terms of the number of edges (hops).
- **Depth-First Search (DFS):** Explores as far as possible along each branch before backtracking.
    - *Concept:* It uses a **LIFO (Last-In-First-Out) Stack** (implemented via recursion in your code).

    - *Outcome:* Useful for checking connectivity, but not guaranteed to find the shortest path.

3. Informed (Heuristic) Search: A* Algorithm:

This is the "intelligent" part of your project. Unlike BFS/DFS, A* uses problem-specific knowledge to find the goal faster.

- **The Evaluation Function:** f(n) = g(n) + h(n)
    - g(n): The actual cost from the start to the current node.
    - h(n): The **Heuristic**—an estimated cost from the current node to the goal.
- **Admissibility:** Your code uses straight-line distance estimates. Since a straight line is always the shortest possible distance, the heuristic is "admissible" (it never overestimates), which guarantees that A* will find the optimal path.

4. Knowledge Representation (KR)

This is the "Brain" of your Smart City. Instead of hard-coding every possibility, you use **Declarative Knowledge**.

- **Fact-Based Representation:** The self.kb dictionary stores facts about the environment (e.g., "Road C to D is blocked during Rush Hour").
- **Rule-Based Reasoning:** The is_path_valid() function acts as an **Inference Engine**. Before the search algorithm moves to a new node, it "queries" the Knowledge Base to see if the move is logically allowed.
- **Dynamic Constraints:** This allows the AI to change its behavior based on "world states" rather than just static map data.

5. Complexity and Data Structures

To make the AI efficient, the code utilizes specific computer science structures:

- **Priority Queue (Min-Heap):** Used in A* to always expand the node with the lowest f(n) first (O(log n) complexity).
- **Adjacency List:** An efficient way to represent sparse graphs (like city maps) in memory.

SCREENSHOTS:

OUTPUT:

```
Data\Local\Programs\Python\Python312\python.exe "c:/Users/ritus/OneDrive/Documents/web dev/web dev projects/self-assisment/python/Fundamenta
l of AI & ML - vityarthi/smartcity.py"
---  Welcome to the Smart City Logic Navigator ---
Available Locations: A, B, C, D

Enter Start Location (e.g., A): A
Enter Destination (e.g., D): C
Is it currently Rush Hour? (yes/no): yes

---  AI Processing Paths ---
 BFS (Fewest Stops): ['A', 'C']
 A* Optimal Path (Traffic Logic Applied): ['A', 'C'] | Total Cost: 2

Try another route? (y/n): n
Goodbye! Safe travels.
PS C:\Users\ritus\OneDrive\Documents\web dev\web dev projects\self-assisment\python\Fundamental of AI & ML - vityarthi>
```

CONCLUSION:

The **Smart City Logic Navigator** successfully demonstrates the integration of classical search algorithms with symbolic knowledge representation. By developing this system, several key objectives in Artificial Intelligence and Machine Learning were achieved like hybrid AI architecture , efficiency of informed search , dynamic adaptability ,